



RAT-Backend — API-Dokumentation

Remote Access Topologie · FastAPI-Backend · REST + JWT

Diese Dokumentation beschreibt die HTTP-Schnittstelle, mit der der C#-Client (`RAT_Data.DatabaseConnection`) spricht. Das Backend ist eine FastAPI-Anwendung (`RAT-Backend/src`). Eine interaktive **Swagger-UI** liegt unter `/docs`, das rohe OpenAPI-Schema unter `/openapi.json`.

 **Basis-URL:** `http://<ip>:<port>` — Standard im Client: `http://127.0.0.1:8000`

 **Auth:** OAuth2-Password-Flow. `POST /user/login` liefert ein JWT-Bearer-Token, das bei jeder weiteren Anfrage als `Authorization: Bearer <token>` mitgeschickt wird.

 **Token-Lebensdauer:** 30 Minuten (der Client erneuert es automatisch).

 **Content-Type:** JSON — **außer** `POST /user/login` (das nutzt `application/x-www-form-urlencoded`, OAuth2-Formular).

 **Hinweis zu PUT/DELETE:** Die meisten Update-/Delete-Endpunkte melden Erfolg, indem sie `HTTPException(200)` *werfen* — eine erfolgreiche Änderung liefert also ****HTTP 200** mit leerem / `{"detail": ...}`-Body, **nicht das geänderte Objekt. POST liefert 201**** mit dem erzeugten Objekt.

Fehlercodes (allgemein)

Status	Bedeutung
400	Validierung fehlgeschlagen. Body: <code>{"status": "validation_error", "errors": [...]}</code>
401	Token fehlt/ungültig/abgelaufen — oder falscher Benutzername/Passwort beim Login
403	Angemeldet, aber nicht berechtigt (zu niedrige Rechtestufe / nicht Owner)

404	ID existiert nicht — auch für ein NetworkObject, das man nicht sehen darf (man soll nicht einmal erfahren, dass es existiert)
409	Eindeutigkeits-Konflikt, z. B. Benutzername oder Geräte-Name bereits vergeben

Rechte-Modell

Jedes NetworkObject hat **pro Benutzer** eine Berechtigungs-Zeile mit einer Stufe (0..4).

Keine Zeile = Stufe 0 (Hidden). Ein **globaler Admin** (`is_admin`) hat implizit `Owner`

auf allem.

Stufe	Name	Bedeutung
0	Hidden	Gerät gar nicht sichtbar (wird herausgefiltert / als 404 gemeldet)
1	See	Gerät + Interfaces sehen
2	Edit	Einstellungen/Interfaces/Verbindungen ändern (nicht löschen)
3	Admin	Edit + niedrigeren Nutzern Hidden/See/Edit vergeben (kein Admin/Owner, keine Admins/Owners ändern)
4	Owner	Admins/Owner ändern, Owner vergeben/entziehen, Gerät löschen

Authentifizierung & Benutzer — `/user`

POST `/user/login` • **(Formular, ohne Auth)**

Body (`application/x-www-form-urlencoded`):

```
username=admin&password=admin
```

200 →

```
{ "access_token": "eyJhbGciOi...", "token_type": "bearer" }
```

401 bei falschen Zugangsdaten.

 Eine frische Datenbank legt automatisch einen Admin an: ****Benutzer** `admin`, Passwort

`admin` ** — bitte sofort ändern.

`GET /user/me`

Liefert den angemeldeten Benutzer:

```
{ "id": 1, "username": "admin", "is_admin": true, "can_create": true }
```

`GET /user/`

Listet alle Benutzer (jeder Angemeldete). Array im Format von `/user/me`. Passwörter werden nie zurückgegeben.

`POST /user/register`

```
{ "username": "alice", "password": "Geheim123", "is_admin": false, "can_create": true }
```

Regeln / Grenzen

- `username` — Pflicht, **eindeutig**, max. **50** Zeichen → `409` falls vergeben.
- `password` — Pflicht.  **Passwort-Richtlinie:** mind. **8 Zeichen**, **mind. 1 Buchstabe** und **mind. 1 Ziffer** (sonst `400`).
- `is_admin` — optional `bool`, Standard `false`.
- `can_create` (Alias `canCreate`) — optional `bool`, Standard `false`. Nötig, um NetworkObjects anzulegen.

201 → erzeugter Benutzer. Eine `UserSettings`-Zeile wird automatisch angelegt.

`PUT /user/{id}`

Alle Felder optional (leeres/fehlendes `password` = unverändert).

```
{ "username": "alice2", "password": "NeuesPw1", "is_admin": false, "can_create": true }
```

- **Globaler Admin** darf jeden + alle Felder ändern.
- **Normaler Benutzer** nur sich selbst, und nur `username` + `password` (sonst `403`).
- Ein neues `password` muss erneut die Passwort-Richtlinie erfüllen.
- Neuer `username` muss eindeutig bleiben (`409`).

DELETE /user/{id}

Löscht einen Benutzer. **Nur für globale Admins** (`is_admin`), sonst `403` .

- Man kann das **eigene Konto nicht löschen** → `400` .
- Unbekannte `id` → `404` .
- Räumt abhängige Zeilen auf: zuerst **Logins** und **SNMP-Einstellungen** an den

Berechtigungs-Zeilen des Nutzers, dann dessen **Berechtigungen** und seine

UserSettings, danach der Benutzer selbst.

200 bei Erfolg (`{ "detail": "User deleted" }`).



Benutzereinstellungen — /user/settings

Eine Einstellungs-Zeile pro Benutzer (automatisch angelegt).

GET /user/settings/

```
{ "zoom": 100, "show_ports": false, "show_interfaces": false }
```

PUT /user/settings/

```
{ "zoom": 150, "show_ports": true, "show_interfaces": false }
```

- `zoom` — `int` (Client nutzt 50–300, Standard 100).
- `show_ports`, `show_interfaces` — `bool` .

200 bei Erfolg.



Netzwerkgeräte — /networkObject

Ein Gerät auf der Arbeitsfläche. Sichtbarkeit ist rechtegesteuert (man bekommt nur Geräte mit Stufe `>= See`).

Objektform (`NetworkObjectOut`)

```
{
  "id": 12, "name": "Core-Router", "type": "Router",
  "x": 320, "y": 220,
  "os": "", "cpu": "", "gpu": "", "ram": "", "specs": ""
}
```

Feld-Grenzen

- `name` — Pflicht, **eindeutig** (409), max. **50** Zeichen.
- `type` — Pflicht, max. **20** Zeichen (Freitext). Client nutzt: `PC`, `Router`, `Switch`, `Server`, `Client`, `Hub`, `Cloud`, `AccessPoint`.
- `x`, `y` — Pflicht- `int` (Koordinaten).
- `os`, `cpu`, `gpu`, `ram` — Pflicht-Strings, je max. **100** Zeichen (`""` falls unbekannt).
- `specs` — Pflicht-String (Freitext, keine Längengrenze).

Methode	Endpunkt	Recht	Antwort
GET	/networkObject/	— (nur sichtbare)	Liste
POST	/networkObject/	<code>can_create</code> (Ersteller wird Owner)	201 + Objekt
PUT	/networkObject/{id}	Edit	200
DELETE	/networkObject/{id}	Owner (löscht kaskadierend Interfaces, Verbindungen, Logins/SNMP, Rechte)	200



Interfaces — /networkObjectInterface

Eine Netzwerkschnittstelle (NIC) eines Geräts.

Objektform

```
{
  "id": 5, "network_object_id": 12, "network_object_connection_id": null,
  "name": "eth0", "max_speed": 1000, "is_up": true,
  "ipv4": "192.168.1.1", "ipv6": "", "ipv4_subnet_mask": "255.255.255.0",
  "ipv6_prefix_length": 0, "ipv4_gateway": "192.168.1.254"
}
```

Feld-Grenzen

- `network_object_id` — Pflicht (zugehöriges Gerät).
- `network_object_connection_id` — **optional / nullable** (`null` , wenn nicht verkabelt).
- `name` — Pflicht, max. **50** Zeichen.
- `max_speed` — Pflicht- `int` (Mbit/s im Client).
- `is_up` — Pflicht- `bool` .
- `ipv4` , `ipv4_subnet_mask` , `ipv4_gateway` — Pflicht-Strings, max. **15** Zeichen.
- `ipv6` — Pflicht-String, max. **45** Zeichen.
- `ipv6_prefix_length` — Pflicht- `int` (sinnvoll `0..128`).

Methode	Endpunkt	Recht	Antwort
GET	/networkObjectInterface/	— (nur sichtbare)	Liste
POST	/networkObjectInterface/	Edit auf <code>network_object_id</code>	201
PUT	/networkObjectInterface/{id}	Edit auf altes + neues Gerät	200
DELETE	/networkObjectInterface/{id}	Edit	200

Verbindungen (Kabel) — /networkObjectConnection

Eine Verbindung zwischen zwei Interfaces.

Objektform (`NetworkObjectConnectionOut`)

```
{ "id": 3, "name": "Kablex", "speed": 1000000000, "type": "Wired", "note": "" }
```

Feld-Grenzen

- `name` — Pflicht, max. **50** Zeichen.
- `speed` — Pflicht- `int` (Bit/s im Client).
- `type` — Pflicht, max. **20** Zeichen. Client nutzt `Wired` / `Wireless` .
- `note` — Pflicht-String (Freitext).

Methode	Endpunkt	Recht	Antwort
GET	/networkObjectConnection/	— (nur sichtbare)	Liste
POST	/networkObjectConnection/	Edit auf beide Endpunkt-Geräte	201
PUT	/networkObjectConnection/{id}	Edit auf beide	200
DELETE	/networkObjectConnection/{id}	Edit auf beide	200

POST-Body (mit den zwei Interface-IDs):

```
{ "name": "Kablex", "speed": 1000000000, "type": "Wired", "note": "",  
  "n01": 5, "n02": 9 }
```



Berechtigungen — /networkObjectPermission

Rechststufe eines Benutzers auf einem Gerät.

Objektform (NetworkObjectPermissionOut)

```
{ "id": 7, "network_object_id": 12, "user_id": 3, "permissions": 3 }
```

- `network_object_id` — Pflicht.
- `permissions` — Pflicht-`int`, $0 \leq x \leq 4$ (validiert; außerhalb → 400).

GET /networkObjectPermission/

Eigene Zeilen + auf Objekten, auf denen man Admin/Owner ist, alle Zeilen. Admins: alle.

POST /networkObjectPermission/ • **201 (Upsert)**

```
{ "network_object_id": 12, "target_user_id": 3, "permissions": 2 }
```

- `target_user_id` — Pflicht: für **wen** das Recht gilt.
- Stufe `0` (Hidden) → stattdessen DELETE benutzen.
- **Regeln:** Admin/Owner nötig. Ein Admin darf nur `0..2` an *niedrigere* Nutzer vergeben; nur ein Owner darf Admin/Owner setzen/ändern. Sonst 403. Bestehende (Nutzer, Objekt)-Zeile wird aktualisiert (keine Duplikate).

PUT /networkObjectPermission/{id} • **200**

Nur `permissions` ändern (gleiche Regeln). Objekt/Nutzer wechseln ist verboten (400).

DELETE /networkObjectPermission/{id} • **200**

Zeile entfernen (= Hidden). Gleiche Admin/Owner-Regeln.



Geräte-Logins — /login

SSH/SFTP/SCP/Telnet-Zugangsdaten, **pro Benutzer** an der eigenen Berechtigungs-Zeile gespeichert.

Objektform (LoginOut)

```
{ "id": 4, "network_object_permission_id": 7, "port": 22,
  "type": "SSH", "username": "root", "password": "secret" }
```

Feld-Grenzen

- `network_object_permission_id` — Pflicht; muss eine **eigene** Berechtigungs-Zeile sein (403).
- `port` — Pflicht- `int` (z. B. 22).
- `type` — Pflicht, max. **10** Zeichen. Client: `SSH`, `Telnet`, `SFTP`, `SCP`.
- `username`, `password` — Pflicht-Strings, je max. **50** Zeichen.

Methode	Endpunkt	Recht	Antwort
GET	/login/	eigene	Liste
POST	/login/	eigene Berechtigungs-Zeile	201
PUT	/login/{id}	eigene (alt + neu)	200
DELETE	/login/{id}	eigene	200



SNMP-Einstellungen — /snmpSettings

Read-/Write-Communities, pro Benutzer an einer Berechtigungs-Zeile.

Objektform (SNMPSettingsOut)

```
{ "id": 2, "network_object_permission_id": 7,
  "read_community": "public", "write_community": "private" }
```

Feld-Grenzen

- `network_object_permission_id` — Pflicht; muss **eigene** Zeile sein (403).
- `read_community`, `write_community` — Pflicht-Strings, je max. **50** Zeichen.

GET / POST (**201**) / PUT (**200**) / DELETE (**200**) — jeweils nur auf eigenen Zeilen.



Schnellreferenz — Wertgrenzen

Feld	Grenze
User.username	≤ 50 Zeichen, eindeutig, Pflicht
User.password	🔒 ≥ 8 Zeichen, ≥ 1 Buchstabe, ≥ 1 Ziffer
NetworkObject.name	≤ 50 Zeichen, eindeutig, Pflicht
NetworkObject.type	≤ 20 Zeichen
NetworkObject.os/cpu/gpu/ram	je ≤ 100 Zeichen
Interface.name	≤ 50 Zeichen
Interface.ipv4 / subnet / gateway	je ≤ 15 Zeichen
Interface.ipv6	≤ 45 Zeichen
Interface.ipv6_prefix_length	int, sinnvoll 0–128
Connection.name	≤ 50 Zeichen
Connection.type	≤ 20 Zeichen
Permission.permissions	int, 0 ≤ x ≤ 4 (erzungen)
Login.type	≤ 10 Zeichen
Login.username / password	je ≤ 50 Zeichen
SNMP.read_community / write_community	je ≤ 50 Zeichen
Token-Lebensdauer	30 Minuten



Die meisten String-Grenzen sind VARCHAR -Größen der Datenbank. Aktiv von der API geprüft

werden die **Passwort-Richtlinie** (≥ 8 Zeichen, Buchstabe + Ziffer) und permissions (0–4).

Andere Bereiche (z. B. ein Mindestwert wie age ≥ 1) sind dokumentiert, wo der Client darauf

baut — daher zusätzlich client-seitig validieren.